

Nama : Raffan Fatih Zulfan
NIM : J0403251091
Kelas : A2

1. Penjelasan Node dan Edge

Pada studi kasus jaringan pertemanan media sosial ini, pemodelan *graph* dibangun menggunakan komponen berikut:

1. Node (Vertex): Mewakili entitas pengguna tunggal di dalam media sosial. Pada *graph* ini, terdapat 5 *node* yang merepresentasikan 5 akun pengguna, yaitu: Ihsan, Raxen, Ivan, Aulia, dan Ibra.
2. Edge (Sisi/Garis): Mewakili status hubungan pertemanan antar pengguna. Terdapat 6 *edge* yang menyambungkan antar *node*, yaitu: Ihsan-Raxen, Ihsan-Ivan, Ihsan-Ibra, Raxen-Aulia, Ivan-Aulia, dan Aulia-Ibra.
3. Karakteristik Graph: Graph ini merupakan Undirected Graph (graph tak berarah) karena hubungan di media sosial ini bersifat *mutualan* (timbang balik). Jika Ihsan berteman dengan Raxen, maka secara otomatis Raxen juga berteman dengan Ihsan.

2. Analisis Adjacency List vs Matrix

Dalam mengimplementasikan struktur pertemanan di atas ke dalam kode Python, terdapat perbedaan performa dan kegunaan antara Adjacency List dan Adjacency Matrix:

2.1 Adjacency List:

- Kelebihan: Sangat hemat memori (*Space complexity* $O(V+E)$). Pada hasil *output*, List hanya menyimpan data teman yang relevan saja (misal: Raxen hanya mencatat Ihsan dan Aulia).
- Kecocokan: Sangat cocok untuk representasi media sosial di dunia nyata yang bersifat Sparse Graph (graph renggang), di mana satu pengguna tidak berteman dengan seluruh pengguna yang terdaftar di platform tersebut.

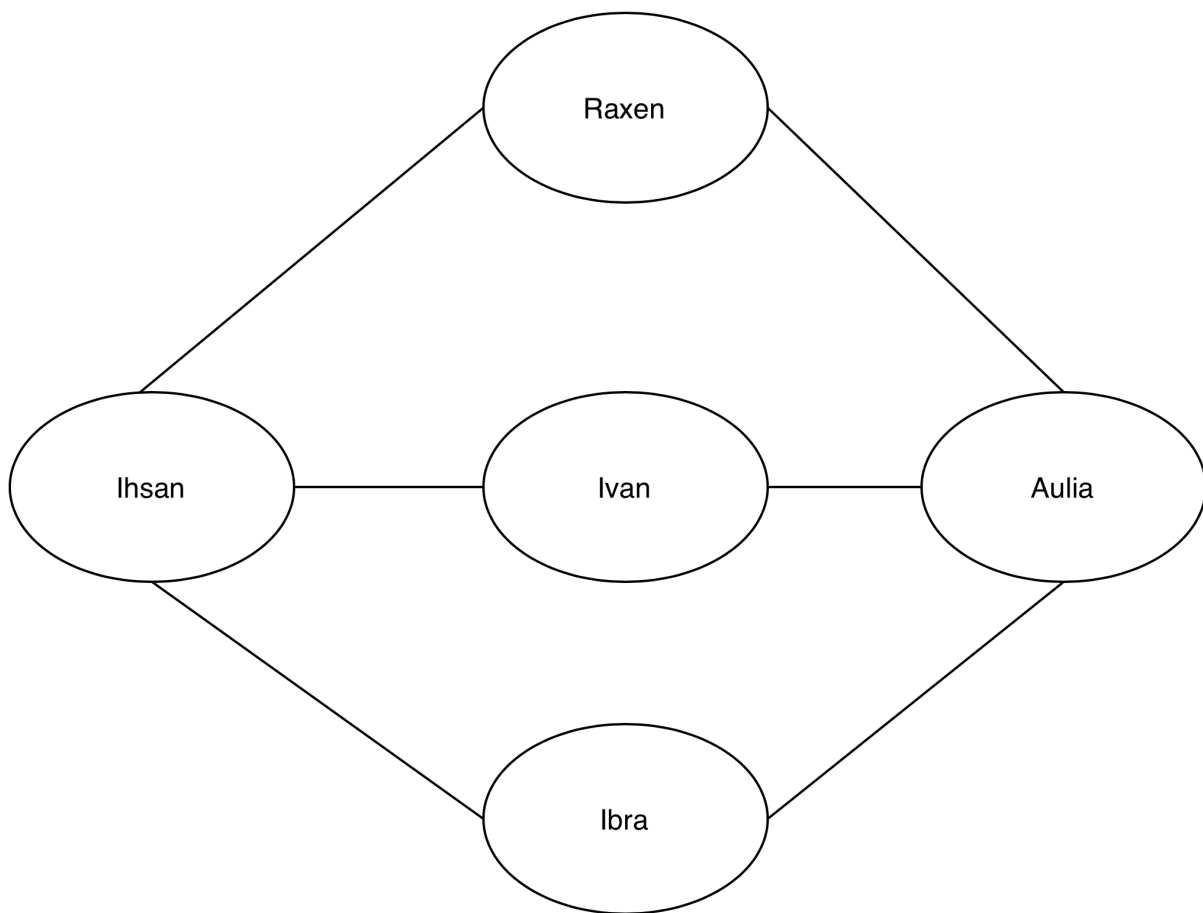
2.2 Adjacency Matrix:

- Kelebihan: Pengecekan hubungan sangat cepat (*Time complexity* $O(1)$). Jika sistem ingin mengecek "apakah Ivan berteman dengan Ibra?", program hanya perlu melihat indeks baris dan kolom matriks tanpa perlu melakukan *looping*.
- Kelemahan: Sangat boros memori (*Space complexity* $O(V^2)$). Pada *output* matriks 5x5, terlihat banyak memori terbuang untuk menyimpan angka 0 (yang berarti tidak berteman).

3. Kesimpulan Singkat

Struktur data *graph* merupakan metode yang sangat efektif untuk memodelkan jaringan sosial. Berdasarkan hasil praktikum, meskipun *Adjacency Matrix* menawarkan kecepatan dalam pencarian relasi spesifik antar dua akun, penggunaan *Adjacency List* lebih direkomendasikan untuk sistem pertemanan media sosial. Hal ini dikarenakan *Adjacency List* lebih efisien dalam mengelola memori (*memory-efficient*) pada struktur jaringan pengguna yang bersifat *sparse* (renggang).

4. Gambar Graph



5. Output Kode

Py

Daftar Node (User):

- Ihsan
- Raxen
- Ivan
- Aulia
- Ibra

Hubungan (Mutualan):

Ihsan - Raxen
Ihsan - Ivan
Ihsan - Ibra
Raxen - Aulia
Ivan - Aulia
Aulia - Ibra

Adjacency List:

Ihsan : ['Raxen', 'Ivan', 'Ibra']
Raxen : ['Ihsan', 'Aulia']
Ivan : ['Ihsan', 'Aulia']
Aulia : ['Raxen', 'Ivan', 'Ibra']
Ibra : ['Ihsan', 'Aulia']

Adjacency Matrix:

[0, 1, 1, 0, 1]
[1, 0, 0, 1, 0]
[1, 0, 0, 1, 0]
[0, 1, 1, 0, 1]
[1, 0, 0, 1, 0]